

Ex. No. 1B	MANAGING TABLES USING DML, DCL AND TCL COMMANDS
Date of Exercise	24/07/2026

Aim

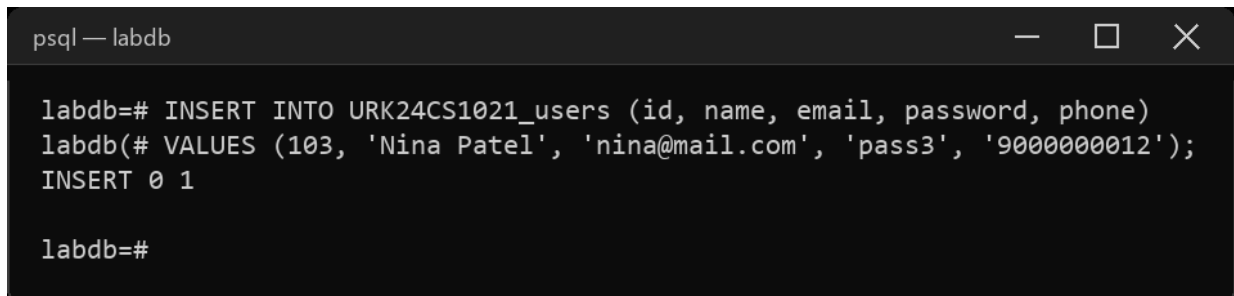
To execute all commands of Data Manipulation Language, Data Control Language, and Transaction Control Language to get the desired output.

Description

DML statements (INSERT/UPDATE/DELETE) manipulate row data. DCL statements (GRANT/REVOKE) control access privileges on database objects for specific roles. TCL statements (COMMIT/ROLLBACK/SAVEPOINT) control transaction boundaries — COMMIT makes changes permanent, SAVEPOINT marks a point to roll back to within an open transaction, and ROLLBACK TO undoes work back to that point without discarding the whole transaction.

1. Insert a new user into the "User" table.**Sample Code:**

```
INSERT INTO URK24CS1021_users (id, name, email, password, phone)
VALUES (103, 'Nina Patel', 'nina@mail.com', 'pass3', '9000000012');
```

Sample Output:

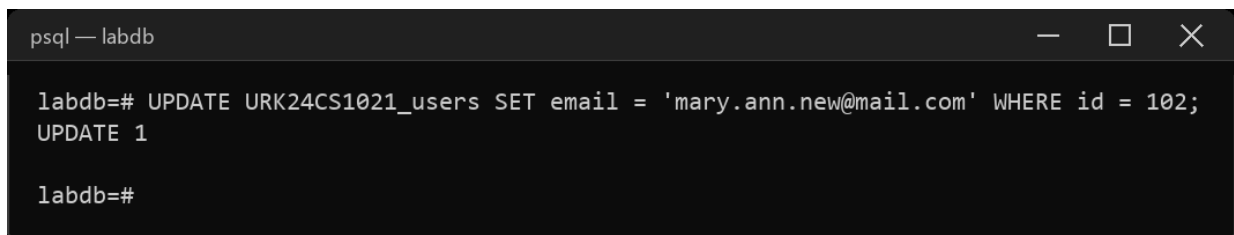
```
psql — labdb

labdb=# INSERT INTO URK24CS1021_users (id, name, email, password, phone)
labdb(# VALUES (103, 'Nina Patel', 'nina@mail.com', 'pass3', '9000000012');
INSERT 0 1

labdb=#
```

2. Update the email address of a user with UserID 102.**Sample Code:**

```
UPDATE URK24CS1021_users SET email = 'mary.ann.new@mail.com' WHERE id = 102;
```

Sample Output:

```
psql — labdb

labdb=# UPDATE URK24CS1021_users SET email = 'mary.ann.new@mail.com' WHERE id = 102;
UPDATE 1

labdb=#
```

3. Delete a user with the email "example@example.com".**Sample Code:**

```
INSERT INTO URK24CS1021_users (id, name, email, password, phone)
VALUES (104, 'Temp User', 'example@example.com', 'pass4', '9000000013');
DELETE FROM URK24CS1021_users WHERE email = 'example@example.com';
```

Sample Output:

```
psql — labdb

labdb=# INSERT INTO URK24CS1021_users (id, name, email, password, phone)
labdb(# VALUES (104, 'Temp User', 'example@example.com', 'pass4', '9000000013');
INSERT 0 1

labdb=# DELETE FROM URK24CS1021_users WHERE email = 'example@example.com';
DELETE 1

labdb=#
```

4. Insert a new event into the "Event" table.**Sample Code:**

```
INSERT INTO URK24CS1021_event (eventid, name, eventdate, eventtime, venueid,
description)
VALUES (402, 'Food Festival', '2023-11-05', '2023-11-05 12:00:00', 1, 'Street
food festival');
```

Sample Output:

```
psql — labdb

labdb=# INSERT INTO URK24CS1021_event (eventid, name, eventdate, eventtime, venueid, description)
labdb(# VALUES (402, 'Food Festival', '2023-11-05', '2023-11-05 12:00:00', 1, 'Street food festival');
INSERT 0 1

labdb=#
```

5. Update the description of an event with EventID 401.**Sample Code:**

```
UPDATE URK24CS1021_event SET description = 'Annual comic convention, now with a
cosplay contest'
WHERE eventid = 401;
```

Sample Output:

```
psql — labdb

labdb=# UPDATE URK24CS1021_event SET description = 'Annual comic convention, now with a cosplay contest'
labdb(# WHERE eventid = 401;
UPDATE 1

labdb=#
```

6. Grant SELECT privileges on the "User" table to a user named "john".**Sample Code:**

```
CREATE ROLE john LOGIN;
GRANT SELECT ON URK24CS1021_users TO john;
```

Sample Output:

```
psql — labdb

labdb=# CREATE ROLE john LOGIN;
CREATE ROLE

labdb=# GRANT SELECT ON URK24CS1021_users TO john;
GRANT

labdb=#
```

7. Revoke INSERT privileges on the "Event" table from a user named "mary".**Sample Code:**

```
CREATE ROLE mary LOGIN;
GRANT INSERT ON URK24CS1021_event TO mary;
REVOKE INSERT ON URK24CS1021_event FROM mary;
```

Sample Output:

```
psql — labdb

labdb=# CREATE ROLE mary LOGIN;
CREATE ROLE

labdb=# GRANT INSERT ON URK24CS1021_event TO mary;
GRANT

labdb=# REVOKE INSERT ON URK24CS1021_event FROM mary;
REVOKE

labdb=#
```

8. Create a new user with the username "jane" and grant them all privileges on the "Ticket" table.**Sample Code:**

```
CREATE ROLE jane LOGIN;
GRANT ALL PRIVILEGES ON URK24CS1021_ticket TO jane;
```

Sample Output:

```
psql — labdb

labdb=# CREATE ROLE jane LOGIN;
CREATE ROLE

labdb=# GRANT ALL PRIVILEGES ON URK24CS1021_ticket TO jane;
GRANT

labdb=#
```

9. Grant EXECUTE privileges on a stored procedure to a role named "admin".

Sample Code:

```
CREATE ROLE admin;
CREATE OR REPLACE PROCEDURE URK24CS1021_ticket_count() LANGUAGE sql AS $$ SELECT
1; $$;
GRANT EXECUTE ON PROCEDURE URK24CS1021_ticket_count() TO admin;
```

Sample Output:

```
psql — labdb

labdb=# CREATE ROLE admin;
CREATE ROLE

labdb=# CREATE OR REPLACE PROCEDURE URK24CS1021_ticket_count() LANGUAGE sql AS $$ SELECT 1; $$;
CREATE PROCEDURE

labdb=# GRANT EXECUTE ON PROCEDURE URK24CS1021_ticket_count() TO admin;
GRANT

labdb=#
```

10. Revoke all privileges from a user named "guest" on all tables in the schema.

Sample Code:

```
CREATE ROLE guest LOGIN;
REVOKE ALL PRIVILEGES ON ALL TABLES IN SCHEMA public FROM guest;
```

Sample Output:

```
psql — labdb

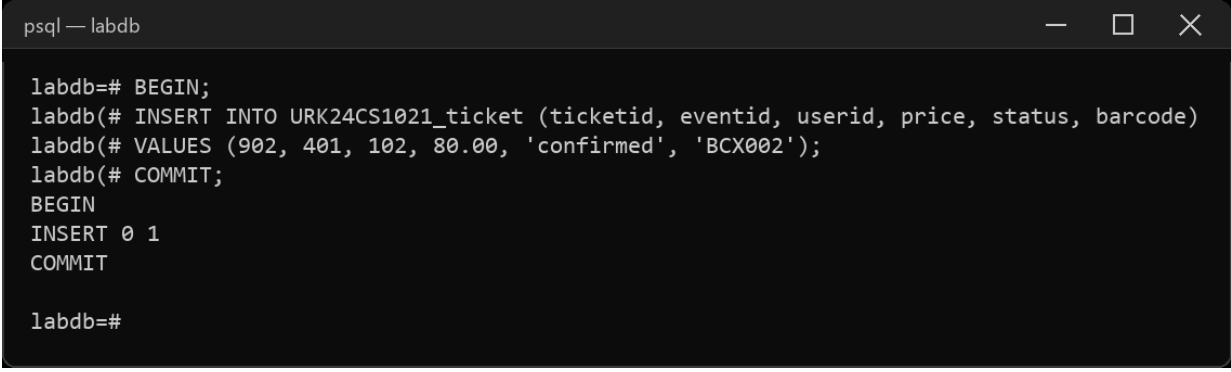
labdb=# CREATE ROLE guest LOGIN;
CREATE ROLE

labdb=# REVOKE ALL PRIVILEGES ON ALL TABLES IN SCHEMA public FROM guest;
REVOKE

labdb=#
```

11. Perform commit a transaction in the database.**Sample Code:**

```
BEGIN;
INSERT INTO URK24CS1021_ticket (ticketid, eventid, userid, price, status,
barcode)
VALUES (902, 401, 102, 80.00, 'confirmed', 'BCX002');
COMMIT;
```

Sample Output:

```
psql — labdb

labdb=# BEGIN;
labdb(# INSERT INTO URK24CS1021_ticket (ticketid, eventid, userid, price, status, barcode)
labdb(# VALUES (902, 401, 102, 80.00, 'confirmed', 'BCX002');
labdb(# COMMIT;
BEGIN
INSERT 0 1
COMMIT

labdb=#
```

12. Perform roll back a transaction to a specific savepoint.**Sample Code:**

```
BEGIN;
INSERT INTO URK24CS1021_ticket (ticketid, eventid, userid, price, status,
barcode)
VALUES (903, 401, 103, 95.00, 'confirmed', 'BCX003');
SAVEPOINT sp1;
INSERT INTO URK24CS1021_ticket (ticketid, eventid, userid, price, status,
barcode)
VALUES (904, 401, 104, 95.00, 'confirmed', 'BCX004');
ROLLBACK TO SAVEPOINT sp1;
COMMIT;
SELECT ticketid FROM URK24CS1021_ticket WHERE ticketid IN (903, 904);
```

Sample Output:

```
psql — labdb

labdb=# BEGIN;
labdb(# INSERT INTO URK24CS1021_ticket (ticketid, eventid, userid, price, status, barcode)
labdb(# VALUES (903, 401, 103, 95.00, 'confirmed', 'BCX003'));
labdb(# SAVEPOINT sp1;
labdb(# INSERT INTO URK24CS1021_ticket (ticketid, eventid, userid, price, status, barcode)
labdb(# VALUES (904, 401, 104, 95.00, 'confirmed', 'BCX004'));
labdb(# ROLLBACK TO SAVEPOINT sp1;
labdb(# COMMIT;
labdb(# SELECT ticketid FROM URK24CS1021_ticket WHERE ticketid IN (903, 904);
BEGIN
INSERT 0 1
SAVEPOINT
INSERT 0 1
ROLLBACK
COMMIT
 ticketid
-----
      903
(1 row)

labdb=#
```

13. Perform set a savepoint within a transaction.

Sample Code:

```
BEGIN;
INSERT INTO URK24CS1021_ticket (ticketid, eventid, userid, price, status,
barcode)
VALUES (905, 401, 105, 70.00, 'confirmed', 'BCX005');
SAVEPOINT sp2;
SELECT COUNT(*) AS tickets_so_far FROM URK24CS1021_ticket;
COMMIT;
```

Sample Output:

```
psql — labdb

labdb=# BEGIN;
labdb(# INSERT INTO URK24CS1021_ticket (ticketid, eventid, userid, price, status, barcode)
labdb(# VALUES (905, 401, 105, 70.00, 'confirmed', 'BCX005');
labdb(# SAVEPOINT sp2;
labdb(# SELECT COUNT(*) AS tickets_so_far FROM URK24CS1021_ticket;
labdb(# COMMIT;
BEGIN
INSERT 0 1
SAVEPOINT
 tickets_so_far
-----
              4
(1 row)

COMMIT

labdb=#
```

14. Enable autocommit mode in the database.

Sample Code:

```
\set AUTOCOMMIT on
```

Sample Output:

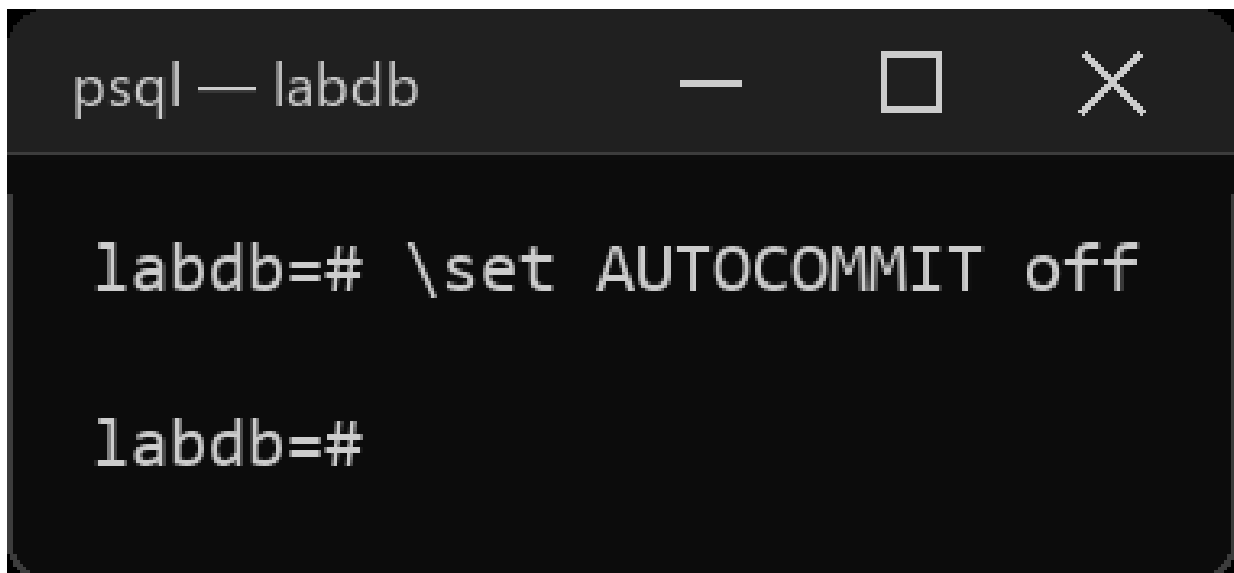
```
psql — labdb

labdb=# \set AUTOCOMMIT on

labdb=#
```

15. Disable autocommit mode in the database.**Sample Code:**

```
\set AUTOCOMMIT off
```

Sample Output:

```
psql — labdb

labdb=# \set AUTOCOMMIT off

labdb=#
```

RESULT

The SQL queries were executed and the desired output was obtained.